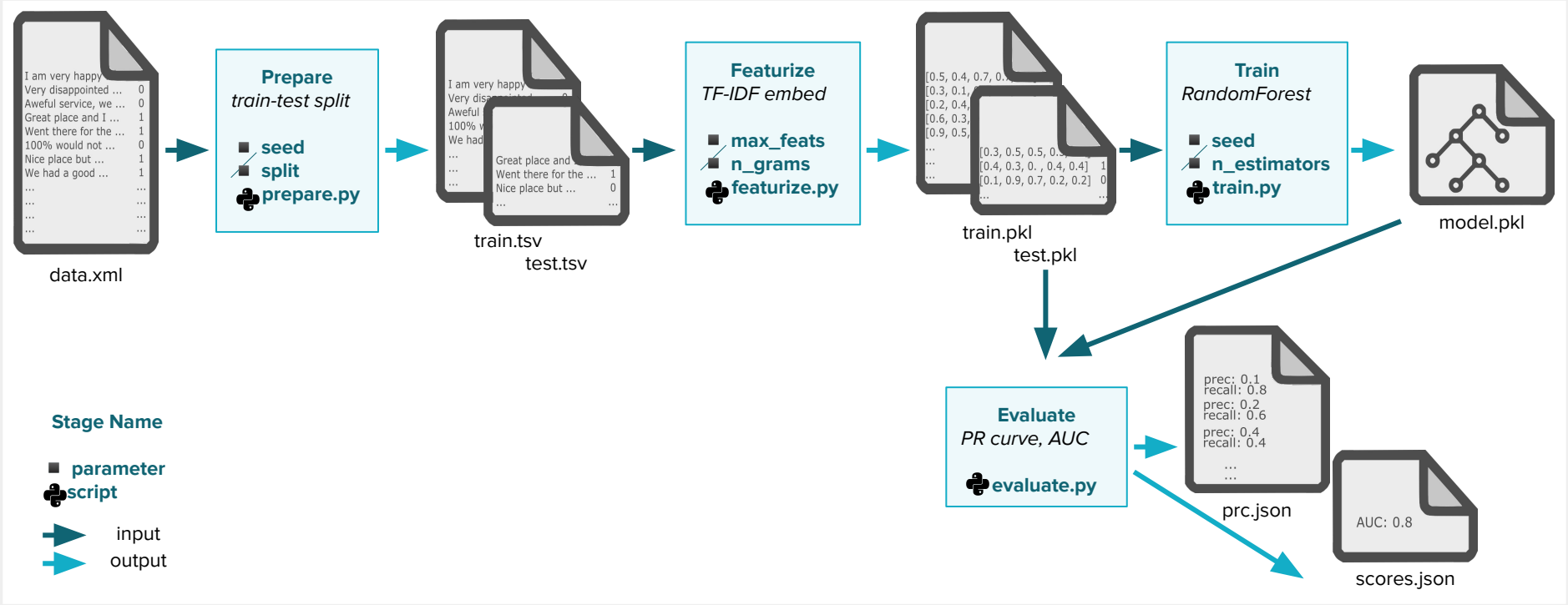
DVC features:
Data and ML pipelines automation

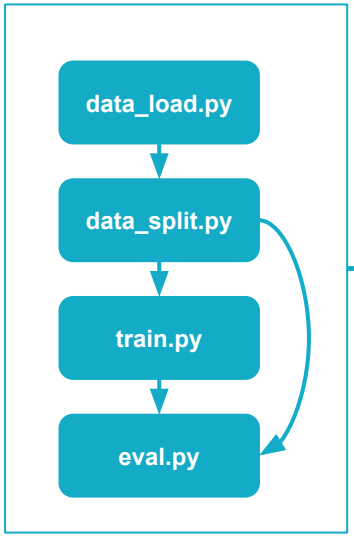
Motivation

- With `dvc add` we can track large files—this includes files such as: trained models, embeddings, etc. However, we also want to **track how such files were generated** for reproducibility and better tracking!

The following is an example of a **typical ML pipeline**. Its structure is a **DAG** (**direct acyclic graph**).



Configure pipelines in a simple dvc.yaml



dvc.yaml M

dvc.yaml > { } stages > { } data_load > cmd

dvc.yaml (schema.json)

```yaml
stages:
  data_load:
    cmd: python src/stages/data_load.py --config=params.yaml
    deps: ...
    params: ...
    outs: ...
  data_split:
    cmd: python src/stages/data_split.py --config=params.yaml
    deps: ...
    params: ...
    outs: ...
  train:
    cmd: python src/stages/train.py --config=params.yaml
    deps: ...
    params: ...
    outs: ...
    plots: ...
  evaluate:
    cmd: python src/stages/eval.py --config=params.yaml
    deps: ...
    params: ...
    outs: ...
    metrics:
      - metrics.json:
          cache: false
```

Load Data

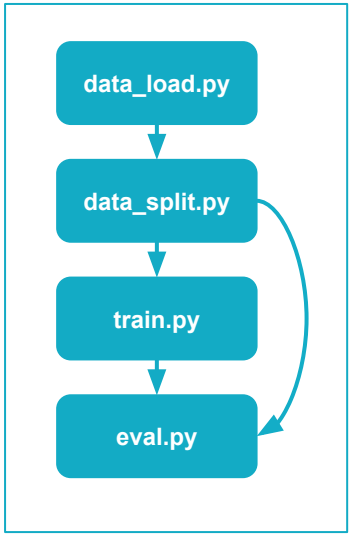
Split Data

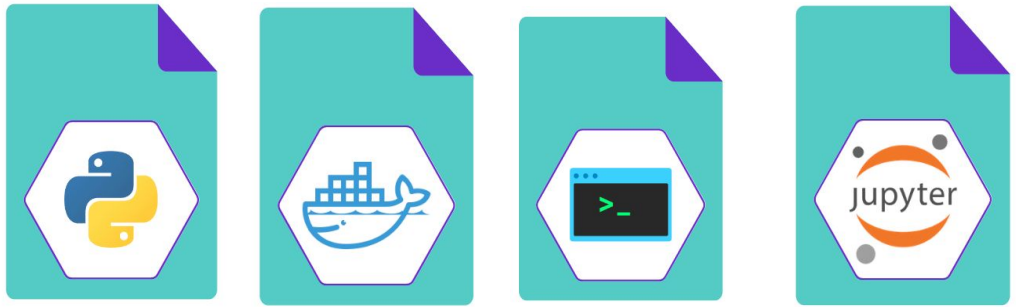
Train Model

Evaluation

Source: Alex Kim, Optimizing Image Segmentation Projects with DVC, Iterative.ai

Use any executable script as a stage job





Python module

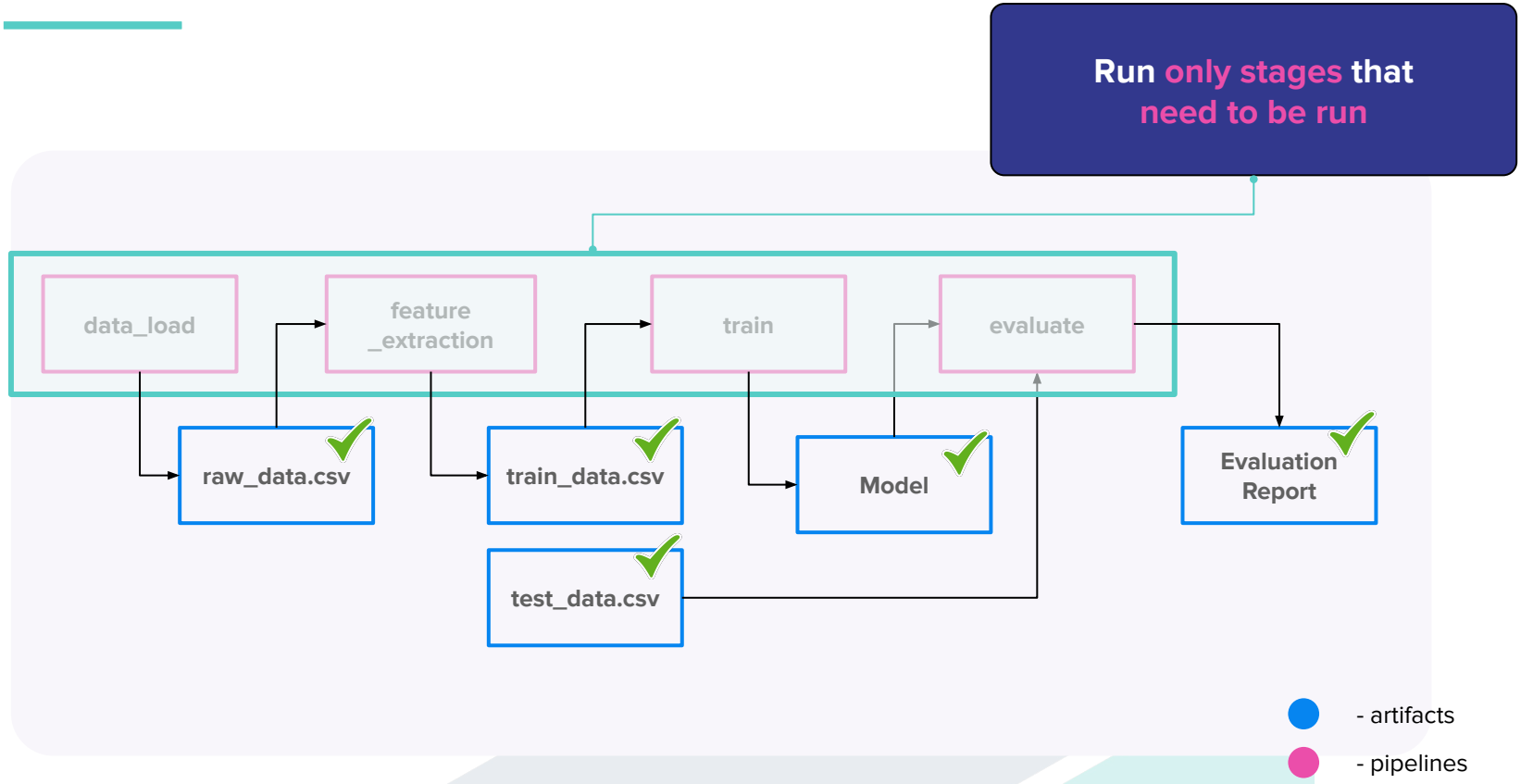
Docker container

Any script (bash)

Jupyter Notebook

Notebooks can be run via "nbconvert"# Convert t

Run as simple as: dvc exp run



Tracking ML Pipelines

- **Option A**: run pipeline stages, then track output artifacts with **`dvc add`**

```
$ python   src/prepare.py   data/data.xml
$ dvc add  data/prepared/train.tsv  data/prepared/train.tsv
```

instead of 'dvc run', read it as 'dvc stage add --run -v -f'

- **Option B**: run pipeline stage and track them together with all dependencies with **`dvc run`**

```
$ dvc run -n prepare \
          -p prepare.seed \
          -p prepare.split \
          -d src/prepare.py \
          -d data/data.xml \
          -o data/prepared \
          python src/prepare.py data/data.xml
```

stage name

parameters — read from `params.yaml`

dependencies (including script!)

outputs to track

```
prepare:
 split: 0.20
 seed: 20170428

featurize:
 max_feats: 500
 ngrams: 1

      ...
```

→ **Advantages of Option B**

1. outputs are automatically tracked (i.e. saved in `.dvc/cache`)
2. pipeline stages with parameters names are saved in **`dvc.yaml`**
3. deps, params, outs are all hashed and tracked in **`dvc.lock`**
4. **like a `Makefile`**, can reproduce by **`dvc run prepare`**—re-run only if deps changed!

Example

- Let's create a DVC repo for an NLP project.

```
$ mkdir nlp_project & cd nlp_project
$ git init & dvc init & git commit -m "Init dvc repo"
```

- Then we download some data + some code to prepare the data and train/evaluate a model

```
$ dvc get https://github.com/iterative/dataset-registry get-started/data.xml \
          -o data/data.xml
$ dvc add data/data.xml
$ git add data/.gitignore data/data.xml.dvc & git commit -m "Add data, first version"
$ wget https://code.dvc.org/get-started/code.zip
$ unzip code.zip & rm -f code.zip
```

```
$ tree
.
├── data
│   ├── data.xml
│   └── data.xml.dvc
├── params.yaml
└── src
    ├── evaluate.py
    ├── featurization.py
    ├── prepare.py
    ├── requirements.txt
    └── train.py
```

YAML file with params for all the stages

pipeline steps

```
prepare:
  split: 0.20
  seed: 20170428

featurize:
  max_features: 500
  ngrams: 1

train:
  seed: 20170428
  n_estimators: 50
```

Example

Use "dvc stage add --run -v -f" instead!!

- Let's run the `prepare` stage.

```
$ dvc run -n prepare \
          -p prepare.seed \
          -p prepare.split \
          -d src/prepare.py \
          -d data/data.xml \
          -o data/prepared \
          python src/prepare.py data/data.xml

$ git add  data/.gitignore  dvc.yaml  dvc.lock
```

```
stages:
 prepare:
   cmd: python src/prepare.py data/data.xml
   deps:
   - data/data.xml
   - src/prepare.py
   params:
   - prepare.seed
   - prepare.split
   outs:
   - data/prepared
```

Describe data pipelines, similar to how `Makefiles` work for building software.

```
prepare:
 cmd: python src/prepare.py data/data.xml
 deps:
 - path: data/data.xml
   md5: a304afb96060aad90176268345e10355
 - path: src/prepare.py
   md5: 285af85d794bb57e5d09ace7209f3519
 params:
   params.yaml:
     prepare.seed: 20170428
     prepare.split: 0.2
 outs:
 - params: data/prepared
   md5: 20b786b6e6f80e2b3fcf17827ad18597.dir
```

Matches the `dvc.yaml` file.

Created and updated by DVC commands like `dvc run`.

Describes latest pipeline state for:

1. track intermediate and final artifacts (like a `.dvc` file)
2. allow DVC to detect when stage defs or dependencies changed, triggering re-run.

- **Note**: dependencies and artifacts are automatically tracked, **no need to `dvc add` them!**

Example

"dvc stage add -n run -v -f " instead

- Then we run the **featurize** and **train** stages in the same way.

```
$ dvc run -n featurize \
          -p featurize.max_features \
          -p featurize.ngrams \
          -d src/featurize.py \
          -d data/prepared \
          -o data/features \
          python src/featurization.py data/prepared data/features

$ git add  data/.gitignore  dvc.yaml  dvc.lock
```

featurization.py

```
$ dvc run -n train \
          -p train.seed \
          -p train.n_estimators \
          -d src/train.py \
          -d data/features
          -o model.pkl
          python src/train.py data/features model.pkl

$ git add  data/.gitignore  dvc.yaml  dvc.lock
```

Example

Use "dvc stage add --run -v -f" instead of "dvc run"

- And finally we run the `evaluation` stage.

```
$ dvc run -n evaluate \
          -d src/evaluate.py \
          -d model.pkl \
          -d data/features \
          --metrics-no-cache scores.json \
          --plots-no-cache prc.json \
          python src/evaluate.py model.pkl data/features scores.json prc.json

$ git add  dvc.yaml  dvc.lock
```

eval/metrics.json

eval/plots

Declare output **plot file.**

A special kind of output file (**-o**), must be JSON and can be used to make comparisons across experiments in a **plot form**.

E.g. here it contains data for **ROC curve plot.**

The `-no-cache` prevents DVC to store the file in cache.

Declare output **metrics file.**

A special kind of output file (**-o**), must be JSON and can be used to make comparisons across experiments in a **tabular form**.

E.g. here it contains data for **AUC score.**

The `-no-cache` prevents DVC to store the file in cache.

Plot dependency graphs

```
$ dvc dag
```



Reproducing Pipelines

- **`dvc repro`** regenerate data pipeline results, by restoring the DAG defined by stages listed in **`dvc.yaml`**. This compares file hashes with **`dvc.lock`** to re-run only if needed. This is **like `make` in software builds.**

- **Case 1: nothing changed**, re-running pipeline stages is skipped.

```
$ dvc repro train
      'data/data.xml.dvc' didn't change, skipping
      Stage 'prepare' didn't change, skipping
      Stage 'featurize' didn't change, skipping
      Stage 'train' didn't change, skipping
      Data and pipelines are up to date.
```

- **Case 2: a dependency changed**, pipeline stages are re-run if needed.

```
$ sed -i -e "s@max_features: 500@max_features: 1500@g" params.yaml
$ dvc repro train
      'data/data.xml.dvc' didn't change, skipping
      Stage 'prepare' didn't change, skipping
      Running stage 'featurize' with command:
            python src/featurization.py data/prepared data/features
      Updating lock file 'dvc.lock'
      Running stage 'train' with command:
            python src/train.py data/features model.pkl
      Updating lock file 'dvc.lock'
      To track the changes with git, run:
            git add dvc.lock
```

Track Experiments in CLI

run "dvc exp run" to create an entry in the experiment tracker

dvc exp show
to visualize metrics

```
$ dvc exp show
```

Experiment	Created	loss	acc	train.epochs	model.conv_units
workspace	-	0.25183	0.9137	10	64
mybranch	Oct 23, 2021	-	-	10	16
├── 9a4ff1c [exp-333c9]	10:40 AM	0.25183	0.9137	10	64
├── 138e6ea [exp-55e90]	10:28 AM	0.25784	0.9084	10	32
└── 51b0324 [exp-2b728]	10:17 AM	0.25829	0.9058	10	16

dvc exp push
to save (commit) experiment

```
$ dvc exp push origin exp-333c9
Pushed experiment 'exp-333c9'to Git remote 'origin'.
```

https://iterative.ai/blog/DVC-VS-Code-extension

Comparing experiments

- **`dvc params diff rev_1 rev_2`** shows how parameters differ in two different git revisions/tags. Without arguments, it shows how they differ in workspace vs. last commit.

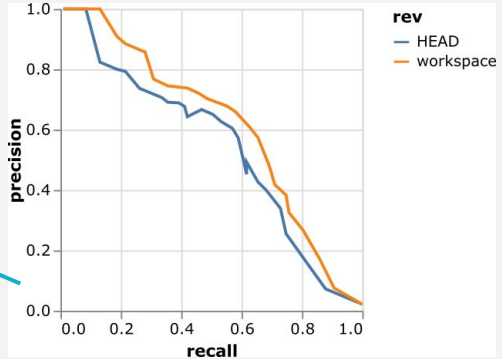
```
$ dvc params diff
Path         Param                   Old    New
params.yaml  featurize.max_features  500    1500
```

- **`dvc metrics diff rev_1 rev_2`** does the same for metrics.

```
$ dvc params diff
Path         Metric    Value    Change
scores.json  auc       0.61314  0.07139
```

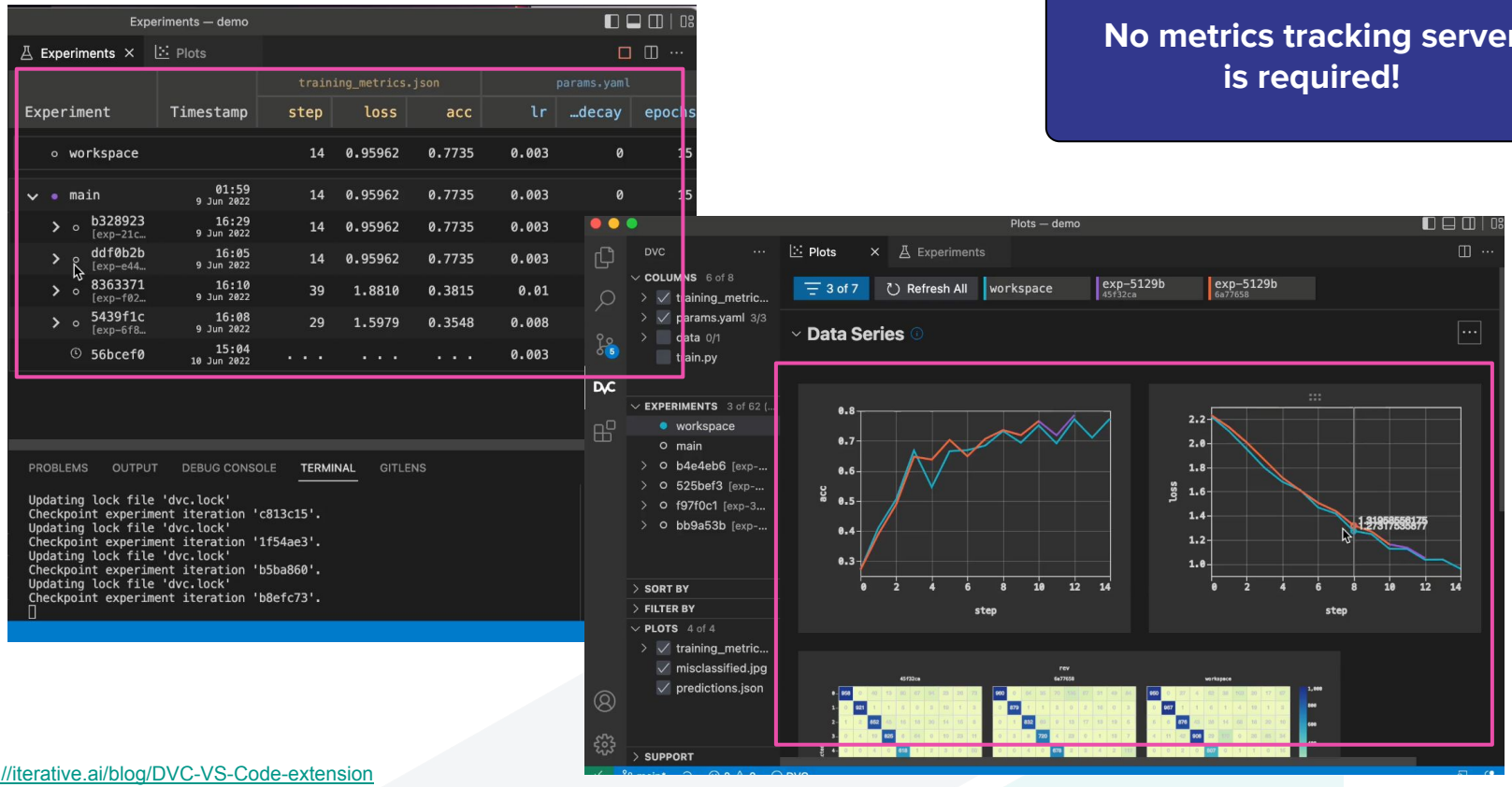
- **`dvc plots diff rev_1 rev_2`** does the same for plots.

```
$ dvc plots diff -x recall -y precision
file:///Users/dvc/example-get-started/plots.html
```



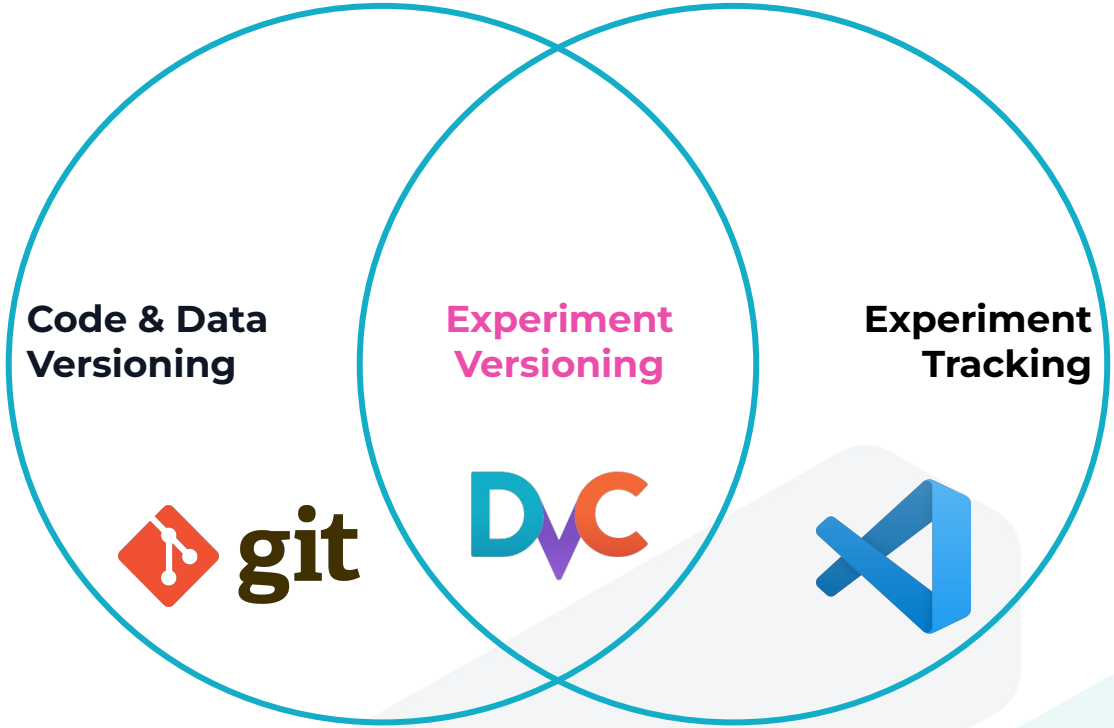
...or, use DVC extension UI in VSCode

No metrics tracking server is required!



https://iterative.ai/blog/DVC-VS-Code-extension

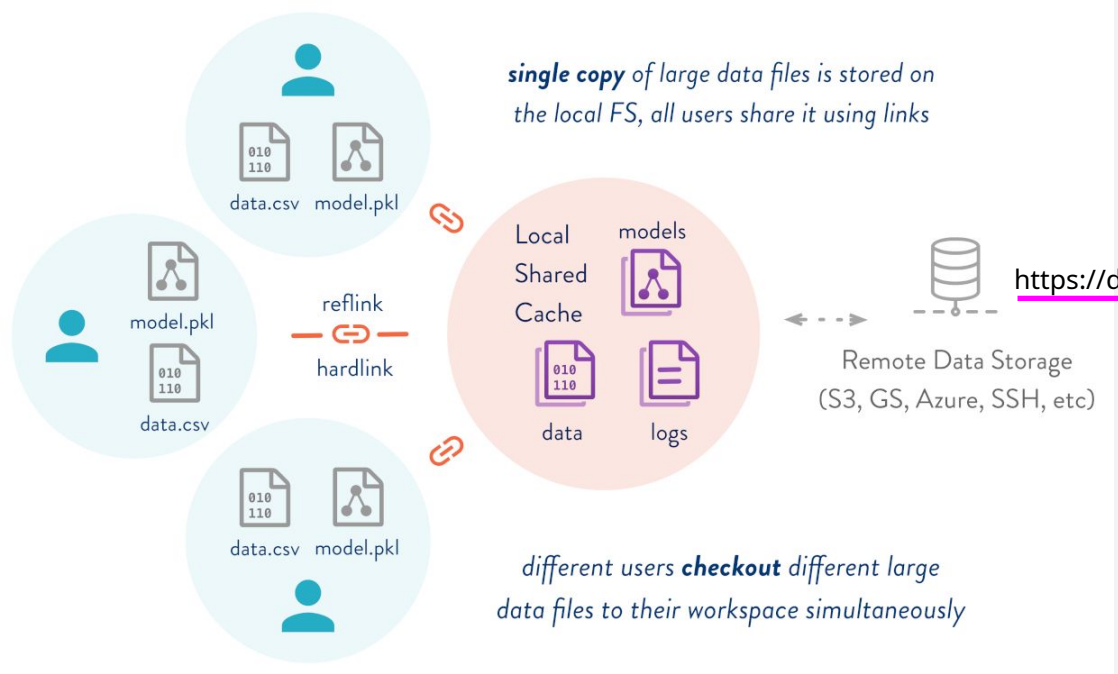
All experiments are versioned





Shared Development Server

Shared Development Server



- **Disk space optimization**
  Avoid having 1 cache per user!

- **Use DVC as usual**
  - Each `dvc add` or `dvc run` moves data to the shared external cache!
  - Each `dvc checkout` links required data to the workspace!

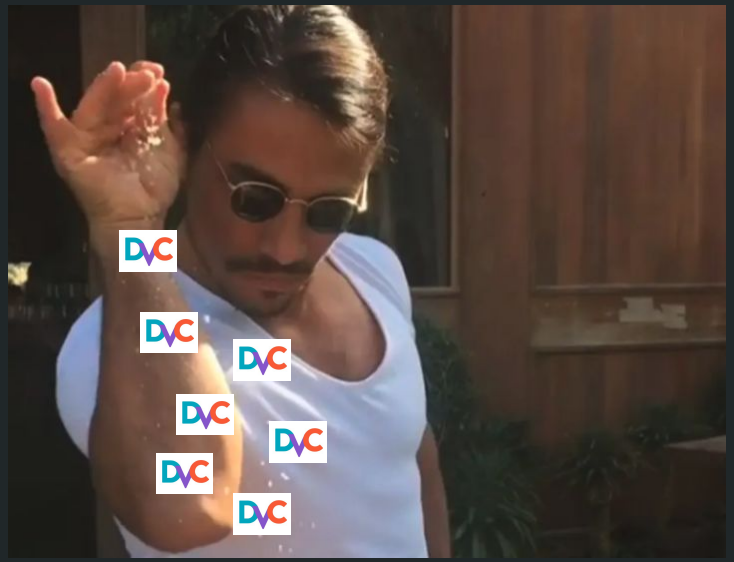
https://doc.dvc.org/user-guide/how-to/share-a-dvc-cache

- See here for implementation details, but basically it's not too difficult:

```
$ mkdir -p path_shared_cache/
$ mv .dvc/cache/* path_shared_cache/

$ dvc cache dir path_shared_cache/
$ dvc config cache.shared group

$ git commit -m "config shared cache"
```



Conclusions

Conclusions

- DVC is a **version control** system for **large ML data and artifacts**.
- DVC **integrates with Git** through **`*.dvc`** and **`dvc.lock`** files, to version files and pipelines, respectively.
- DVC repos can work as **data registries**, i.e. a middleware between cloud storage and ML projects
- To **track raw ML data files**, use **`dvc add`**—e.g. for input dataset.
- To **track intermediate or final results** of a ML pipeline, use **`dvc run`**—e.g. for model weights, dataset.
- Consider using a **shared development server** with a **unified, shared external cache**